WEEK 1 · NOTES & ASSIGNMENT

Decomposition. How developers actually think.

The single biggest mindset shift between a syntax writer and a real developer — installed in one session.

CORE SKILL	FRAMEWORK	ASSIGNMENT
Think First, Code Second	**Inputs → Process → Output → Edges**	**10 Logic Problems**

Rajdeep Dhakad
Founder · Code Easy Academy

01
WEEK ONE

PART 01 · THE MINDSET

Why You're Really Here

> *You've watched 100+ hours of tutorials. You can read JavaScript. You know loops, arrays, functions. But ask you to build something from scratch — and your mind goes blank.*

This marathon is **not about JavaScript**. It's about how you think. JavaScript is a language. Knowing the language doesn't make you a writer. **Problem-solving makes you a developer.**

THE TRUTH NOBODY TOLD YOU

Professional developers spend more time thinking than typing. The code on the screen is the LAST step, not the first. The real work happens BEFORE you open VS Code — in your head, on paper, in plain English. JavaScript is just the language we use to write down a solution we already figured out.

Two Students. Same Knowledge. Different Outcomes.

STUDENT A · SYNTAX-FIRST

Builds nothing.

```
function calculateTip() {
    // ...uhh...
    let bill =
    // wait, what am I doing?
    // do I need an array?
    // should this be a class?
}
```

20 minutes pass. Nothing written. Quits.

STUDENT B · LOGIC-FIRST

Builds the app.

```
// Step 1: Get bill amount
// Step 2: Get tip percentage
// Step 3: tip = bill × (% / 100)
// Step 4: total = bill + tip
// Step 5: Show tip and total

// NOW I know what to build.
```

5 min thinking saved 30 min confusion.

THE ONE THING THAT MATTERS

This is the only difference between you and a developer right now. Not talent. Not years of experience. Just **this habit**.

PART 02 · THE FRAMEWORK

The 4-Step Decomposition

Every program — from a tip calculator to Instagram — does the same three things at its core: it takes something IN, does something WITH IT, and gives something OUT. **Once you see this pattern, you can break down any problem.**

The IPO + Edges Model

01
INPUTS
What information do I need to start?

02
PROCESS
What do I DO with that information?

03
OUTPUT
What do I want to give back?

04
EDGES
What could go wrong?

Applied to: The Tip Calculator

STEP	QUESTION	ANSWER
Inputs	What do I need to start?	Bill amount, tip percentage
Process	What do I DO with it?	Multiply bill by (tip% / 100)
Output	What do I give back?	Tip amount, total bill
Edges	What could go wrong?	Bill is 0, negative, text instead of number

MEMORIZE THIS. TATTOO IT.

Every problem in this marathon — and every problem in your career — fits into this template. **Inputs. Process. Output. Edges.**

KEY MOMENT

Pause here. **Write** these 4 questions in your notebook. Not type — write. Studies show writing by hand creates 2× stronger memory than typing. This is the foundation of everything for the next 6 weeks.

PART 03 · THE WALKTHROUGH

The Tip Calculator, End to End

This is what "thinking like a developer" actually looks like. Three stages — and notice we don't open VS Code until the very end.

Stage A · Apply the Framework (Plain English)

Open a blank text file. Talk through every line out loud. Let yourself *hear* the thinking.

PLAIN-ENGLISH DECOMPOSITION

```
PROBLEM: Build a tip calculator.

INPUTS:
  • Bill amount (number)
  • Tip percentage (10, 15, or 20)

PROCESS:
  • Convert tip % to decimal (15 → 0.15)
  • Multiply bill by decimal → tip amount
  • Add tip to bill → total

OUTPUT:
  • Tip amount
  • Total bill

EDGES:
  • Bill is 0 → show 0 tip, 0 total
  • Bill is negative → show error
  • Tip % is 0 → tip is 0, total = bill
```

Stage B · Translate to Pseudocode

Only after thinking is done. Closer to code, still no real JavaScript.

PSEUDOCODE

```
GET bill from user
GET tipPercent from user

IF bill is less than 0:
    SHOW "Invalid bill"
    STOP

tip = bill * (tipPercent / 100)
```

```
total = bill + tip

SHOW tip
SHOW total
```

Stage C · Now Open VS Code

Each pseudocode line maps to roughly one JavaScript line. The code writes itself.

TIP-CALCULATOR.JS

```javascript
const bill = parseFloat(prompt("Enter bill amount:"));
const tipPercent = parseFloat(prompt("Tip % (10/15/20):"));

if (bill < 0) {
    alert("Invalid bill");
} else {
    const tip = bill * (tipPercent / 100);
    const total = bill + tip;
    alert(`Tip: ₹${tip}, Total: ₹${total}`);
}
```

THE LIGHTBULB MOMENT

Notice something? You never got stuck. You never typed "function" and froze. The code wrote itself **because you knew exactly what you wanted before you started.** THIS is what developers mean when they say "plan first."

PART 04 · THE CODE OF HONOR

3 Rules. Screenshot This.

Mindset alone won't carry you through 6 weeks — but these three reframes will. Print them. Stick them on your wall. This is the spine of the marathon.

RULE ONE 01

OLD MINDSET

"I need to learn more syntax first."

NEW MINDSET

"I need to solve more problems." Skill is built by attempting, not consuming.

RULE TWO 02

OLD MINDSET

"I'll start when I feel ready."

NEW MINDSET

"Ready comes AFTER doing." Confidence is earned through reps, not collected before them.

RULE THREE 03

OLD MINDSET

"I got stuck. I'm not smart enough."

NEW MINDSET

"Stuck = next thing to learn." Every dev was stuck where you are now.

What to Watch For This Week

- [ ] If you feel like writing JS first, then "pseudocode" after as decoration — **stop.** That's the trap.
- [ ] Don't skip EDGES. This is the difference between a junior and a senior dev.
- [ ] Simpler is better. Logic beats cleverness, every single time.
- [ ] If you're stuck, post your **pseudocode** in the group — not your code.

> **REMEMBER**
>
> Week 1 is not about JavaScript. It's about installing a thinking habit. If you leave today writing pseudocode before code, you've succeeded — even if your syntax is shaky, even if you don't finish all problems. **The habit is the win.**

PART 05 · THIS WEEK'S WORK

Week 1 · Exercise Pack

10 problems, scaled from easy to medium-hard. **Pseudocode is mandatory** for every problem. JavaScript is optional this week — focus on the thinking, not the typing.

THE RULE

If you write JavaScript before pseudocode, the answer is wrong — **even if it works.** The whole point is to build the THINKING habit. This week, the process matters more than the result.

01 The Greeting Machine

⭐ EASY

Ask the user for their name and age. Print:

→ "Hi [name], you are [age] years old."

→ "You will be [age + 10] in 10 years."

EDGES empty name, age = 0

02 The Even / Odd Detector

⭐ EASY

Ask for a number. Print "Even" or "Odd". If they enter 0, print "Zero is special."

HINT · *Use % (the remainder operator).*

EDGES 0, negative numbers

03 The Age Group Finder

⭐ EASY

Ask for the user's age. Print which group they belong to:

- → 0–12 → "Child"
- → 13–19 → "Teenager"
- → 20–59 → "Adult"
- → 60 and above → "Senior"

EDGES age = 0, age exactly 12 / 13 / 19 / 20

04 The Bigger Number Finder

⭐ EASY

Ask the user for two numbers. Print which one is bigger. If both are equal, print "They are equal."

EDGES both same, negatives, decimals

05 The Temperature Converter

⭐ EASY

Ask for a temperature in Celsius. Convert to Fahrenheit using:

```
F = (C × 9/5) + 32
```

Output: "X°C = Y°F"

Bonus: tell them if it's "Cold" (below 15°C), "Pleasant" (15–30°C), or "Hot" (above 30°C).

EDGES 0°C, negative temps, very high temps

06 The Age in Days Calculator

⭐⭐ EASY-MEDIUM

Ask for the user's age in years. Output:

- → Age in months
- → Age in weeks
- → Age in days
- → Age in hours

EDGES age = 0, age = 1

07 The Movie Ticket Pricing

⭐⭐ MEDIUM

Ask for age + show time ("matinee" or "evening"). Pricing:

- → Under 12 or over 60 → ₹100 base
- → Everyone else → ₹250 base
- → Matinee → 20% off
- → Evening → full price

Output: base price, discount, final price.

EDGES age exactly 12, age exactly 60, invalid show time

08 The Simple Calculator

⭐⭐ MEDIUM

Ask for two numbers and an operator (+, -, *, /). Perform the operation and print the result.

EDGES dividing by zero, invalid operator (like % or ^)

09 The Electricity Bill Calculator

⭐⭐⭐ MEDIUM-HARD

Ask for units consumed in a month. Calculate bill using slab system:

- → First 100 units → ₹3 per unit
- → Next 100 units (101–200) → ₹5 per unit
- → Next 100 units (201–300) → ₹7 per unit
- → Above 300 units → ₹10 per unit

Output: total bill + breakdown of each slab.

> **EXAMPLE** · *250 units → (100 × 3) + (100 × 5) + (50 × 7) = ₹1,150*

EDGES 0 units, exactly 100/200/300, very large like 500

10 The Restaurant Bill Splitter

⭐⭐ MEDIUM

Ask for:

- → Total bill amount
- → Number of people
- → Whether to add tip (yes/no) — if yes, add 10%

Output: total with tip (if any), per person amount.

EDGES 0 people, 0 bill, just one person

PART 06 · THE PROCESS

How to Submit

Submission Rule

This week, focus on building the **thinking habit**. Code is optional — pseudocode is not.

- [ ] **Part 1 — Pseudocode** in plain English. INPUTS, PROCESS, OUTPUT, EDGES. *Mandatory.*
- [ ] **Part 2 — JavaScript** that mirrors the pseudocode line by line. *Optional for now.*
- [ ] If you're stuck, post your **pseudocode** in the group — never just the code.
- [ ] Friday Logic Lab — every problem walked through live.

Pseudocode Template (Use This Format)

FOR EVERY PROBLEM

```
INPUTS:   ?
PROCESS:  ?
OUTPUT:   ?
EDGES:    ?

Steps:
  1. ...
  2. ...
  3. ...
```

Bonus Challenge (Optional)

DECOMPOSE A REAL APP

Take ANY app you used today — Zomato, Uber, WhatsApp, Instagram. Pick ONE small feature ("sending a message", "calculating delivery fee"). Apply the 4-step framework to it. **You don't need to code it** — just decompose it. Post your decomposition in the group. Best one gets a shoutout in Friday's Logic Lab.

CLOSING THOUGHT

Today you didn't just learn a framework. You learned the difference between someone who copies tutorials and someone who builds. Every problem you'll ever face — in this marathon, in interviews,

in your job — fits into **Inputs → Process → Output → Edges.** Practice it this week. Pseudocode FIRST. Always. See you Friday.

★ END OF WEEK 1 ★